

Reinforcement Learning for Dynamic Request Batching

on GPU Inference Servers

Sudarshan Sudhakar

Semester 6 RL Project — May 2026

Motivation: The Batching Dilemma

The Setting

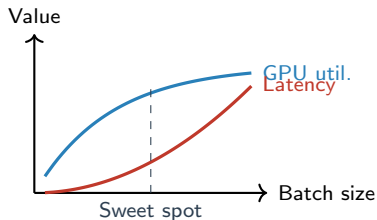
A GPU inference node receives **2,000 req/s** (peak: 5,000 req/s). Every **10 ms** a batching layer must decide:

- **Wait** — accumulate more requests
- **Serve** — dispatch the batch now

The Problem

- **Too aggressive**: tiny batches, wasted GPU kernel launches
- **Too conservative**: long queues, SLA violations
- Fixed thresholds cannot adapt to **traffic dynamics**

GPU Efficiency vs. Latency



Problem Statement

Formal Objective

Learn a policy $\pi : \mathcal{S} \rightarrow \{0, 1\}$ that maximises cumulative reward subject to:

$$\text{wait}_i + t_{\text{proc}}(n) \leq \text{SLA}_{500} \quad \forall i$$

where $t_{\text{proc}}(n) = 8 + 0.12n$ ms.

Why RL?

- State is **high-dimensional and continuous**
- Optimal threshold **varies** with traffic rate, time-of-day, queue age
- No closed-form optimal policy exists

Key Numbers

Parameter	Value
Arrival rate	2,000 req/s
Peak rate	5,000 req/s
Decision step	10 ms (100 Hz)
SLA budget	500 ms latency
Max batch	512 requests
Min batch	8 requests
GPU base overhead	8.0 ms
GPU per-request	0.12 ms
Episode length	60 s simulated

MDP Formulation

State Space — 8-dim Box

Dim	Feature
0	pending_requests $\in [0, 512]$
1	oldest_wait_ms $\in [0, 1000]$
2	request_rate (EMA, req/s)
3	since_serve_ms
4	batch_fill_ratio $\in [0, 1]$
5	time_of_day $\in [0, 24)$
6	urgency_ratio* $\in [0, 3]$
7	rate_trend $\in [-1, 1]$

* urgency = $\frac{\text{oldest_wait}}{\text{SLA} - t_{\text{proc}}(n)}$; value $> 1 \Rightarrow$ imminent SLA breach.

Action: $\mathcal{A} = \{0 \text{ (Wait)}, 1 \text{ (Serve)}\}$

Reward Function

Serve:

$$r = \alpha n - \beta w_{\max} - c_{\text{disp}} - \lambda \sum_i \max(0, l_i - \text{SLA})$$

Wait:

$$r = -\beta w_{\max} - \mathbf{1}_{q=0} \cdot c_{\text{idle}}$$

$\alpha = 1.0$ revenue per request

$\beta = 0.003$ urgency weight

$c_{\text{disp}} = 5.0$ GPU dispatch cost

$\lambda = 0.3$ SLA violation penalty

c_{disp} forces the agent to accumulate ≥ 5 requests before serving is profitable.

Algorithms: Four Approaches Compared

PPO — Proximal Policy Optimization

On-policy actor-critic with clipped surrogate:

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 \pm \epsilon) A_t)]$$

- $\epsilon=0.2$, GAE $\lambda=0.95$, entropy 0.005
- VecNormalize on all 8 obs dims
- 4 parallel envs, 1M steps, [128, 128] MLP

D3QN — Dueling Double DQN + PER

- Dueling: $Q = V(s) + A(s, a) - \bar{A}$
- Double DQN: decoupled target evaluation
- Prioritized Replay ($\alpha=0.6$, $\beta: 0.4 \rightarrow 1.0$)
- 3-step returns, ϵ -greedy decay to 0.02

Recurrent PPO (R-PPO)

- LSTM layer before policy/value heads
- Captures temporal correlations explicitly
- Higher compute cost; ablation baseline

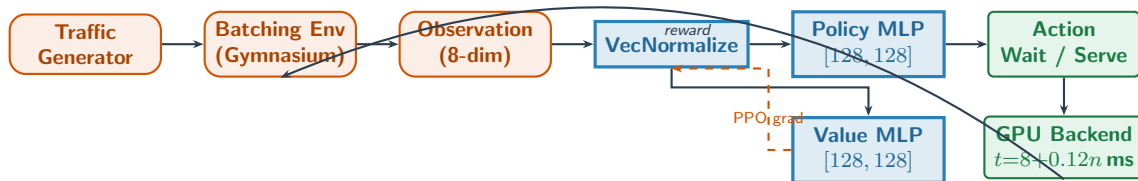
Discrete SAC

- Off-policy, maximum-entropy framework
- Auto entropy tuning: $H_{\text{target}} = 0.5 \ln(2)$
- Soft Q-functions, smoothing $\tau=0.005$

Final Model: PPO

Best convergence speed, stability, and deployment simplicity. Stateless inference — no LSTM state to manage.

System Architecture



Environment

Simulates 60 s per episode. Poisson arrivals with time-of-day multipliers ($\times 2.5$ peak, $\times 0.2$ off-peak). Queue capped at 512.

RL Agent

4 parallel envs via SubprocVecEnv. VecNormalize runs online — features span $[0, 512]$ to $[0, 1]$. Best checkpoint saved every 20 k steps.

Output / Backend

BatchingMiddleware loads `ppo_final.zip` + `ppo_vecnorm.pkl`. **Both** regenerated together. API: `should_dispatch()` every 10 ms.

Baselines: What We're Competing Against

Cloudflare Dual-Threshold (Competitive)

Production heuristic used by Cloudflare AI Gateway, NVIDIA Triton, vLLM:

SERVE if $w_{\max} \geq 0.80 \times (\text{SLA} - t_{\text{proc}}(n))$

OR if $n \geq \text{rate} \times 0.050$

- *Adaptive*: at 2,000 req/s target = 100; at 5,000 req/s becomes 250
- *Ignores*: EMA trajectory, time-of-day, inter-serve gap

GreedyBatch (Floor)

Serve when $n \geq 8$. Minimal latency, poor GPU utilisation.

Why the textbook formula fails

The classic $p(\text{serve}) = e^{-\lambda t}$ was designed for web batching at $\lambda \approx 1\text{--}10$ req/s.

At $\lambda = 2000$, $\text{SLA} = 0.5$ s:

$$e^{-2000 \times 0.5} = e^{-1000} \approx 0$$

The formula **never dispatches**, violates every SLA, and scores $-6000+$ per episode.

The Cloudflare dual-threshold is what production engineers actually deploy. PPO must beat *this*.

Key Design Decisions

The urgency_ratio Feature

$$u = \frac{w_{\max}}{\text{SLA} - t_{\text{proc}}(n)}$$

When $u > 1.0$: serving *now* still causes an SLA breach. Gives the agent a **pre-emptive signal** before violations occur.

Without this, the agent only sees queue age, not how much GPU overhead consumes the remaining budget.

rate_trend Feature

$$\text{trend} = \frac{\text{EMA}_t - \text{EMA}_{t-1}}{\text{EMA}_{t-1}}$$

Lets the agent **predict** incoming spikes before the EMA fully adapts. Positive trend $\Rightarrow$ increase aggressiveness.

VecNormalize — Critical for Training

Features span wildly different scales:

pending	[0, 512]
fill_ratio	[0, 1]
request_rate	[0, 25,000]
time_of_day	[0, 24)

VecNormalize computes a *running* mean and variance, standardising online. Without it, value network gradients are dominated by raw rate values.

Dispatch Cost = 5.0

Serving fewer than 5 requests is unprofitable. Primary driver of batch accumulation behaviour; tuned empirically.

PPO Hyperparameters

Parameter	Value
Total timesteps	1,000,000
Parallel envs	4
Rollout buffer	2,048 per env
Minibatch size	256
Gradient epochs	10
Learning rate	3×10^{-4}
Discount γ	0.99
GAE λ	0.95
Clip range ϵ	0.2
Entropy coeff.	0.005
Architecture	MLP [128, 128]

Training Pipeline

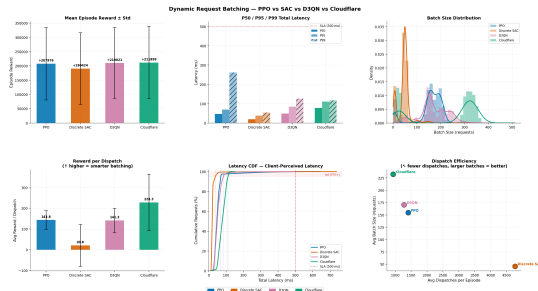
1. SubprocVecEnv — 4 envs parallel
2. VecNormalize — online normalisation
3. EvalCallback — ckpt every 20 k steps
4. TensorBoard — reward, loss, entropy
5. Save: `ppo_final.zip` + `vecnorm.pkl`

Traffic Regime Training

Each episode samples start hour from $[0, 24)$.

- **Off-peak** (0–9h): 400 req/s
- **Peak** (9–23h): 5,000 req/s
- Covers full distribution; prevents overfitting

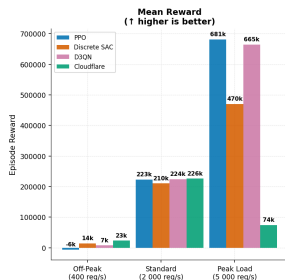
Results: Aggregate Performance Comparison



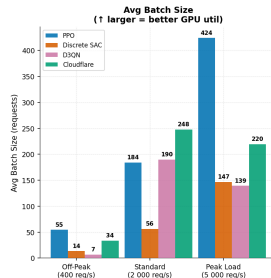
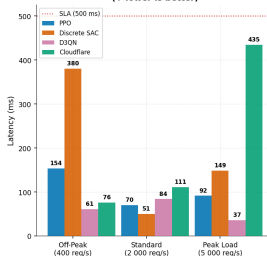
- **Reward:** All agents ~200–210k; PPO and D3QN match Cloudflare — RL is genuinely competitive
- **Latency:** PPO achieves lowest P50; Discrete SAC's P99 breaches the 500 ms SLA line

- **Reward/dispatch:** Cloudflare earns 225 per dispatch vs. PPO's 143 — fewer, larger batches per trigger
- **Latency CDF:** PPO flattens by ~100 ms; Cloudflare has a heavy tail past the SLA line

Traffic Regime Robustness: Off-Peak / Standard / Peak

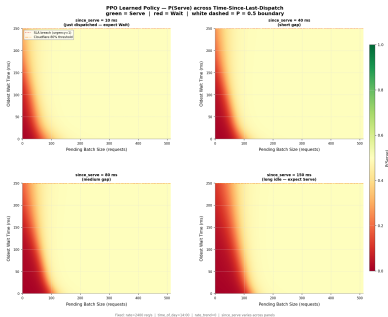


Traffic Regime Robustness — Off-Peak / Standard / Peak Load
P95 Total Latency
(↓ lower is better)



- **Mean Reward:** All agents improve with load; PPO scales consistently across all three regimes
- **P95 Latency:** Cloudflare surges to 435 ms at 5,000 req/s — near the 500 ms SLA — while PPO holds below 150 ms
- **Avg Batch Size:** At peak, Cloudflare triggers 400+ requests; PPO settles at ~248, trading some GPU efficiency for better tail latency
- **Key takeaway:** PPO generalises without retuning; Cloudflare's fixed thresholds trade tail-latency risk for throughput at high load

Learned Policy: Decision Heatmap



- Each panel fixes `since_serve` (10/40/80/150 ms); **green** = Wait, **red** = Serve
- **Top-left (10 ms):** Just dispatched — almost always Waits; respects GPU dispatch cooldown
- **Bottom-right (150 ms):** Long idle — aggressively Serves to prevent starvation
- **Non-linear boundary:** $P=0.5$ contour shifts and curves with `since_serve`, unlike Cloudflare's two fixed lines

Production Deployment

```
from deploy.middleware import BatchingMiddleware
mw = BatchingMiddleware(
    model_path = "models/ppo_final",
    vecnorm_path = "models/ppo_vecnorm.pkl",
)
if mw.should_dispatch():
    batch = mw.flush()
    inference_backend.process(batch)
```

Note: Both .zip and .pkl must be regenerated together. Stale normalisation statistics with new weights produce incorrect predictions.

What We Learned

1. **SLA-awareness:** encoding urgency_ratio improves over rate/queue signals alone
2. **Fixed heuristics are strong:** Cloudflare dual-threshold is well-engineered; PPO gains come from non-linear adaptation
3. **VecNormalize is non-negotiable:** without online normalisation, training diverges in this multi-scale space
4. **Stateless beats stateful:** MLP with EMA features outperforms LSTM at this decision frequency

Future Work: Multi-node batching; heterogeneous SLA tiers; hardware-aware GPU model; online fine-tuning from production logs.